
MEMORIA TÉCNICA Y DOCUMENTACIÓN ACADÉMICA MULTINIVEL: MINECRAFT INVENTORY VIEWER (MIV)

Este documento constituye la memoria descriptiva y el manual de referencia técnica integral para el proyecto "Minecraft Inventory Viewer" (MIV). Ha sido ampliado y detallado minuciosamente de forma exhaustiva para servir como memoria de entrega académica en ciclos formativos y grados universitarios de Ingeniería de Software.

INDICE TEMÁTICO DE LA MEMORIA:

1. RESUMEN EJECUTIVO Y ANÁLISIS DE OBJETIVOS
2. ARQUITECTURA GENERAL E INFRAESTRUCTURA DE TRES NIVELES
3. ANÁLISIS EXHAUSTIVO Y FLUJO DE LOGICA DEL CLIENTE (8 ARCHIVOS FUENTE)
4. ANÁLISIS EXHAUSTIVO DE LA API BACKEND (LAS 16 ENTIDADES, SERVICIOS Y CONTROLLER)
5. ESQUEMA DETALLADO DE PERSISTENCIA (DIAGRAMA ER Y TABLAS SQL / HIBERNATE)
6. MAPA DE ENDPOINTS REST Y PROTOCOLO DE TRANSFERENCIA JSON
7. TECNOLOGÍAS UTILIZADAS EN EL PROYECTO
8. GUÍA DE INSTALACIÓN, INICIALIZACIÓN Y VALIDACIÓN (MANUAL DE PRUEBAS)

1. RESUMEN EJECUTIVO Y ANÁLISIS DE OBJETIVOS

El proyecto Minecraft Inventory Viewer (MIV) es un sistema híbrido que integra un motor de juego 3D interactivo en tiempo real (Minecraft Forge 1.18.2) con un ecosistema empresarial de microservicios REST (Spring Boot 3.x, JPA y Hibernate) y un motor de persistencia relacional SQL (MariaDB).

A través de MIV, un jugador dentro de un servidor de Minecraft puede registrar "bases" geográficas, vincular contenedores físicos (cofres, barriles, shulkers) a dichas bases, rastrear y sincronizar automáticamente sus contenidos en tiempo real mediante hilos paralelos, y abrir un intuitivo y robusto dashboard de control (GUI) presionando la tecla 'B'. Dicho panel permite buscar ítems, visualizar logs de auditoría e interactuar con wikis de información especializada

(bloques, biomas, alimentos y encantamientos) mediante un flujo dinámico de navegación cruzada.

Objetivos Didácticos y Tecnológicos Alcanzados:

1. Programación asíncrona no bloqueante: Separar el hilo de renderizado gráfico ("Render Thread" de OpenGL) de las peticiones HTTP de red de manera que no se experimenten caídas de FPS ("lags" o micro-congelamientos de pantalla).
2. Arquitectura limpia en capas (Clean Architecture): Separación estricta de responsabilidades en el servidor REST mediante Controllers, Services, DTOs, MapStruct Mappers y Repositorios JPA.
3. Consistencia de datos y transaccionalidad: Implementar endpoints masivos atómicos que limpian e insertan inventarios de manera segura en lote.
4. Auditoría y Trazabilidad: Diseñar un motor de registro que almacena un log histórico de cada movimiento de ítems (Inventory History) y logs de actividad de jugadores (Activity Logs).

=====

2. ARQUITECTURA GENERAL E INFRAESTRUCTURA DE TRES NIVELES

=====

El sistema se compone de una infraestructura estructurada en tres niveles físicos e independientes que colaboran de manera desacoplada:

...

[NIVEL 1: Mod Forge Client]

↑ (Llamadas asíncronas HTTP/JSON - CompletableFuture)

[NIVEL 2: Spring Boot REST API]

↑ (Hibernate ORM / Spring Data JPA)

[NIVEL 3: Persistencia MariaDB SQL]

...

FLUJO DINÁMICO DE EJECUCIÓN (Caso de Uso: Apertura y Cierre de un Cofre):

1. El jugador abre un cofre físico en el mundo. El Mod de Minecraft captura esta acción pero no interfiere.
2. El jugador retira o deposita ítems (ej. 32 Diamantes) y cierra la interfaz del cofre.
3. El manejador de eventos `InventoryEventListener` del Mod intercepta el evento `ContainerEvent.Close`.
4. El listener lee de forma segura el inventario físico en memoria (tipo de cofre, coordenadas X, Y, Z, bioma del jugador y lista de stacks de ítems).
5. Se invoca de manera asíncrona a `InventoryApiClient.syncStorage(...)`. Esta llamada corre en un hilo del sistema secundario ajeno a la tasa de fotogramas,

- evitando retrasos.
6. El microservicio Spring Boot recibe la petición JSON en ``StorageController.syncStorage(...)``.
 7. Dentro del servidor, en una sola transacción (``@Transactional``):
 - a. Se verifica la existencia de la base.
 - b. Se busca o registra el bioma y el tipo de contenedor.
 - c. Se limpia la lista de ítems antiguos del cofre para prevenir ítems fantasmas.
 - d. Se agregan los nuevos ítems de forma masiva (batch) vinculando el ID de catálogo global.
 8. La base de datos guarda los cambios y el API REST devuelve un código de estado "200 OK" junto con la información detallada del cofre.
 9. En el cliente mod, al recibirse la confirmación, se actualizan los datos en segundo plano y si el jugador abre el dashboard (tecla 'B'), el cofre y sus ítems aparecen perfectamente actualizados.

3. ANÁLISIS EXHAUSTIVO Y FLUJO DE LÓGICA DEL CLIENTE (8 ARCHIVOS FUENTE)

El mod cliente está implementado en Java 17 sobre la API oficial de Minecraft Forge para la versión 1.18.2. Está ubicado en el paquete ``com.joel.inventoryviewer``. A continuación se detalla técnicamente la lógica interna de cada uno de los 8 archivos del proyecto:

1. InventoryViewer.java

- * Clase principal y punto de partida de la carga del Mod en Minecraft.
- * Lógica Técnica:
 - Utiliza la anotación ``@Mod("inventoryviewer")`` de Forge para registrar el mod.
 - Define un objeto estático ``INSTANCE`` que actúa bajo el patrón Singleton para ofrecer acceso global rápido.
 - En su constructor, asocia los eventos del bus de Forge:
 - * ``FMLClientSetupEvent``: Llama a registrar y enlazar los mapeos de teclado (``KeyInputHandler``) en el juego.
 - * ``RegisterItems``: Registra de forma oficial el ítem personalizado "wand" (Varita de Inventario, instancia de ``WandItem``).
 - * Registra en el bus de eventos común (``MinecraftForge.EVENT_BUS``) la clase ``InventoryEventListener`` para escuchar interacciones de bloques en el mundo.

2. InventoryApiClient.java

-
- * El administrador central de comunicaciones de red del Mod.
 - * Lógica Técnica:
 - Inicializa un ``HttpClient`` estático privado configurado para peticiones HTTP/1.1 y HTTP/2.
 - Define la ruta base ``API_URL = "http://localhost:8080/api"``.
 - Utiliza ``Gson`` para la serialización y deserialización a JSON de las peticiones.
 - ****Asincronía con CompletableFuture:**** Todos sus métodos estáticos son de tipo ``CompletableFuture<T>``. Utilizan ``.sendAsync()`` para enviar peticiones en segundo plano de forma no bloqueante.
 - Métodos implementados:
 - * ``getBases()``: Devuelve ``CompletableFuture<JsonArray>`` con todas las bases de la API.
 - * ``createBase(...)``, ``updateBase(...)``, ``deleteBase(...)``: CRUD de bases.
 - * ``createStorage(...)``: Registra un cofre físico con sus coordenadas X, Y, Z.
 - * ``syncStorage(...)``: Sincroniza en masa el inventario completo de un cofre.
 - * ``logPlayerActivity(...)``: Registra eventos del jugador en los logs del servidor.
 - * ``getPlayerByUuid(...)``: Obtiene la cuenta del jugador usando su identificador único de Mojang.
 - * ``createPlayer(...)``: Registra al jugador por primera vez si no existe en base de datos.
 - Asegura que los endpoints utilicen estrictamente la convención de plurales (ej. ``/player-activity-logs``).

3. InventoryEventListener.java

- * El agente encargado de interceptar eventos in-game del ciclo del inventario.
- * Lógica Técnica:
 - Anotada con ``@Mod.EventBusSubscriber`` para registrar automáticamente métodos escuchadores en Forge.
 - Métodos clave:
 - * ``@SubscribeEvent public static void onRightClickBlock(PlayerInteractEvent.RightClickBlock event)``: Intercepta clics derechos de la varita sobre bloques. Si el jugador tiene la Varita en su mano, previene la apertura por defecto del contenedor y, si la base activa está configurada, registra la posición del cofre (X, Y, Z) de manera asíncrona llamando a ``InventoryApiClient.createStorage(...)``.
 - * ``@SubscribeEvent public static void onContainerClose(ContainerEvent.Close event)``: Se dispara cuando el jugador cierra cualquier contenedor (cofre, barril, etc.). El método recorre todas las ranuras (``slots``) del contenedor abierto, agrupa los ítems por su nombre técnico de registro (ej. ``minecraft:iron_ingot``), suma sus cantidades y despacha de manera asíncrona la lista resultante

a la API mediante `InventoryApiClient.syncStorage(...)` para garantizar que el inventario físico en MariaDB coincida exactamente.

4. KeyInputHandler.java

* Intercepta las entradas del teclado del jugador para abrir las GUIs del Mod.

* Lógica Técnica:

- Declara una variable estática `DASHBOARD_KEY` de tipo `KeyMapping` asignada por defecto a la tecla física 'B'.
- Se suscribe al evento `InputEvent.KeyInputEvent`.
- Si la tecla presionada coincide con `DASHBOARD_KEY` y no hay pantallas abiertas (`Minecraft.getInstance().screen == null`), realiza una consulta de red asíncrona para cargar la información del jugador. Si el jugador no existe en base de datos, lo registra e inicia automáticamente el flujo in-game otorgando la varita y mostrando un mensaje en chat. Acto seguido, levanta la interfaz gráfica principal mediante `Minecraft.getInstance().setScreen(...)`.

5. MainDashboardScreen.java

* La interfaz visual más importante y compleja del Mod (Screen del Dashboard).

* Lógica Técnica:

- Hereda de la clase `Screen` nativa de Minecraft y maneja renderizado directo con `PoseStack` y llamadas OpenGL.
- Implementa un sistema de 8 pestañas (`Tab`) ordenadas de manera estética.
- Integra una barra de búsqueda (`EditBox`) interactiva con la que el usuario puede filtrar el listado mostrado.
- **getFilteredCatalog() (Deduplicación):** Para evitar la carga de ítems duplicados y con textura rota (barrera roja `Items.BARRIER`), filtra el listado global de ítems del catálogo usando un `LinkedHashMap` indexado por el nombre del ítem.
 - Si encuentra una variante con textura válida en lugar de una barrera, la sobrescribe dinámicamente en el mapa. Este listado deduplicado e impecable se comparte para renderizar los elementos en pantalla, calcular clics físicos y determinar la paginación dinámica del catálogo.
- **Navegación Cruzada Dinámica:** Permite hacer clic en cualquier ítem que esté almacenado en un cofre del jugador o en el catálogo global de ítems y, si tiene una entrada relacionada en la base de datos como bloque o alimento, redirige instantáneamente al jugador a la pestaña de "Wikis" (Alimentos o Bloques)

- para ofrecer información del mismo (nutrición, dureza del bloque, etc.).
- Soporta salida rápida e intuitiva presionando la tecla nativa 'E'.

6. SelectionHandler.java

- * Renderiza contornos y resaltes tridimensionales sobre los bloques en el mundo.
- * Lógica Técnica:
 - Maneja la visualización de los cofres asociados a la base del jugador.
 - Se suscribe a `RenderLevelLastEvent` (el último paso del ciclo de renderizado visual de Minecraft).
 - Si hay posiciones registradas y seleccionadas por la varita, toma las coordenadas de la cámara del jugador, calcula la matriz de transformación lineal y dibuja líneas vectoriales 3D brillantes alrededor del cofre en el mundo de juego, ofreciendo una guía visual futurista y premium de los cofres asociados.

7. WandItem.java

- * Representa al objeto físico de la Varita de Inventario dentro del juego.
- * Lógica Técnica:
 - Hereda de `Item` de Minecraft.
 - En `useOn(...)` (clic derecho sobre bloques en el mundo):
 - * Detecta si es un contenedor válido y el jugador tiene permisos.
 - * Si la base está activa, invoca de manera asíncrona a `InventoryApiClient.createStorage(...)` para guardarlo y lo notifica visualmente mediante efectos de partículas.
 - En `use(...)` (clic derecho al aire):
 - * Abre la pantalla simplificada HUD de gestión de bases (`WandScreen`) permitiendo cambios rápidos en pleno juego.

8. WandScreen.java

- * Interfaz rápida HUD de la Varita para cambio rápido de bases activas.
- * Lógica Técnica:
 - Hereda de `Screen`.
 - Dibuja un panel flotante simplificado de bases disponibles para el jugador con una interfaz visual premium de estilo glassmorphism.
 - Cuenta con botones con estados "activar" y "activa" unificados bajo la misma estética y estilo gráfico.
 - Permite salir cómodamente del HUD presionando la tecla 'E'.

=====

4. ANÁLISIS EXHAUSTIVO DE LA API BACKEND (LAS 16 ENTIDADES, SERVICIOS Y CONTROLLER)

El servidor backend está diseñado sobre Spring Boot utilizando una estructura de capas estricta e independiente. A continuación se detallan cada uno de sus 16 componentes funcionales, incluyendo entidades Hibernate, controladores REST pluralizados, mappers MapStruct y lógicas de negocio:

LAS 16 ENTIDADES DE NEGOCIO (Hibernate / JPA Entities):

1. Base.java: Representa una base física o geográfica de un jugador.
 - Campos: ``id`` (PK autoincremental), ``name`` (Nombre único de la base), ``description`` (Descripción detallada de la base), ``createdAt`` e ``updatedAt``.
 - Relaciones: ``@OneToMany(mappedBy = "base", cascade = CascadeType.ALL)`` con ``BaseMember``, ``BaseTag`` y ``Storage``. Esto asegura que si una base se borra, sus miembros, etiquetas y cofres se borren en cascada automáticamente.
2. Player.java: Representa las cuentas de los jugadores registradas en el API.
 - Campos: ``id`` (PK), ``name`` (Nombre de usuario en Minecraft), ``uuid`` (UUID único proporcionado por Mojang), ``activeBaseId`` (ID de la base seleccionada actualmente).
3. Storage.java: Representa un contenedor físico (cofre) registrado.
 - Campos: ``id`` (PK), ``x``, ``y``, ``z`` (Coordenadas físicas tridimensionales), ``base`` (FK a la base propietaria).
 - Relaciones: ``@ManyToOne`` con ``Biome`` (Bioma donde está el cofre) y ``ContainerType`` (Tipo de cofre y número de ranuras). Con ``@OneToMany`` hacia ``StorageItem``.
4. StorageItem.java: Relación intermedia de cantidad de ítems dentro de un cofre.
 - Campos: ``id`` (PK), ``quantity`` (Cantidad total acumulada), ``lastUpdate`` (Fecha y hora).
 - Relaciones: ``@ManyToOne`` con ``Storage`` e ``Item``.
5. Item.java: El catálogo maestro global de ítems disponibles en el servidor.
 - Campos: ``id`` (PK), ``name`` (Nombre de registro técnico, ej: ``minecraft:apple``), ``displayName`` (Nombre legible), ``stackSize`` (Tamaño de pila de ítems, ej: 64).
6. PlayerActivityLog.java: Auditoría de eventos del jugador.

- Campos: `id` (PK), `action` (Descripción de la acción), `timestamp`.
- Relaciones: `@ManyToOne` hacia `Player` (Jugador que ejecutó la acción).

7. InventoryHistory.java: Historial del contenido y variaciones de cofres.

- Campos: `id` (PK), `oldQuantity` (Cantidad antes), `newQuantity` (Cantidad después),
`changeReason` (Motivo del cambio), `changeType` (Tipo de cambio: ADDED, REMOVED, CLEARED),
`timestamp`.
- Relaciones: `@ManyToOne` hacia `Storage`, `Item` y `Player` (actor del cambio).

8. ContainerType.java: Especificaciones técnicas de tipos de cofres.

- Campos: `id` (PK), `name` (ej: `double\_chest`), `slots` (Número de slots), `description`.

9. Biome.java: Catálogo de biomas de Minecraft.

- Campos: `id` (PK), `name` (ej: `minecraft:plains`), `displayName` (ej: Plains).

10. Block.java: Catálogo técnico de bloques (Wiki de bloques).

- Campos: `id` (PK), `name` (Identificador técnico), `displayName`, `hardness` (Dureza), `blastResistance` (Resistencia a explosiones), `toolRequired` (Herramienta).

11. Enchantment.java: Catálogo técnico de encantamientos del juego.

- Campos: `id` (PK), `name`, `displayName`, `maxLevel`, `rarity`, `description`.

12. Food.java: Especificaciones técnicas de alimentos del juego.

- Campos: `id` (PK), `name`, `displayName`, `nutrition` (Puntos de comida),
`saturation` (Saturación), `isMeat` (Si es carne), `fastFood` (Si se consume rápido).

13. BaseMember.java: Relación intermedia de miembros compartidos en bases.

- Campos: `id` (PK), `role` (Rango: OWNER, MEMBER, GUEST), `accepted`,
`permissionsSet`, `note`.
- Relaciones: `@ManyToOne` hacia `Base` y `Player`.

14. BaseTag.java: Etiquetas vinculadas a las bases para su búsqueda rápida.

- Campos: `id` (PK), `tag` (Nombre de etiqueta), `scope` (Ámbito: PUBLIC, PRIVATE), `color`.
- Relaciones: `@ManyToOne` hacia `Base`.

15. ItemEnchantment.java: Relación técnica de encantamientos aplicados a ítems.

- Campos: `id` (PK), `level` (Nivel del encantamiento aplicado).
- Relaciones: `@ManyToOne` hacia `Item` y `Enchantment`.

16. ItemFood.java: Relación técnica que vincula ítems a sus propiedades nutricionales.

- Campos: `id` (PK). Relaciones `@ManyToOne` hacia `Item` y `Food`.

LOS 16 SERVICIOS DE LÓGICA DE NEGOCIO (Service Layer):

Cada una de las 16 entidades cuenta con su respectiva clase de servicio encargada

de coordinar transacciones y proveer métodos CRUD limpios al API. Destacan:

- ****StorageService.java / StorageController.java (Procesador Masivo):****

En lugar de realizar múltiples llamadas lentas para registrar 27 o 54 slots de un cofre, el endpoint `/sync` de `StorageController` ejecuta en un único paso transaccional atómico (`@Transactional`) la limpieza de ítems huérfanos de ese cofre (`storageItemRepository.deleteAll(...)`), agrega los nuevos ítems en un lote y los asocia a sus respectivos registros del catálogo maestro, reduciendo el tráfico de consultas a milisegundos y garantizando que la sincronización de inventario físico sea impecable.

- ****PlayerActivityLogService.java:**** Registra acciones del jugador (como "Crear Base")

inyectando automáticamente la fecha y hora actual del sistema (`LocalDateTime.now()`).

5. ESQUEMA DETALLADO DE PERSISTENCIA (DIAGRAMA ER Y TABLAS SQL / HIBERNATE)

La base de datos relacional utiliza MariaDB 10.x. El mapeo a tablas físicas se realiza mediante anotaciones JPA de Hibernate.

DIAGRAMA DE ENTIDAD-RELACIÓN (ER):

...

[PLAYER] 1 ----- * [PLAYER_ACTIVITY_LOG]

1 1
| |
* *

[BASE_MEMBER] * --- 1 [BASE] 1 ----- * [STORAGE] 1 ----- * [STORAGE_ITEM] * -- 1 [ITEM]

1 1
| |
* *

[PLAYER]

[BASE_TAG]

|
|
*

[ITEM_FOOD]

|
1

[FOOD]

...

La integridad referencial está garantizada mediante:

- `ON DELETE CASCADE` en las relaciones de `BASE` hacia `STORAGE`, `BASE\_MEMBER` y

`BASE\_TAG`. Al borrar una base, MariaDB limpia de forma consistente toda la información ligada en el disco de manera inmediata.

- Índices de base de datos de rendimiento: Un índice compuesto único en la tabla

`STORAGE` en base a las columnas `(base\_id, pos\_x, pos\_y, pos\_z)`. Esto permite

- que la consulta Hibernate `findByBaseIdAndXAndYAndZ` de

`StorageRepository` se ejecute con un costo de búsqueda de $O(1)$ a nivel de base de datos.

6. MAPA DE ENDPOINTS REST Y PROTOCOLO DE TRANSFERENCIA JSON

Toda la comunicación de red del proyecto sigue las directrices REST. Los recursos se pluralizan de la siguiente manera:

ENTIDAD BASE: `/api/bases`

- * GET `/api/bases` - Recupera el listado completo de bases.
- * GET `/api/bases/{id}` - Obtiene detalles de una base por su ID.
- * GET `/api/bases/search?query=..` - Busca bases por coincidencia de texto.
- * POST `/api/bases` - Payload JSON (`name`, `description`) crea una base.
- * PUT `/api/bases/{id}` - Payload JSON actualiza nombre y descripción.
- * DELETE `/api/bases/{id}` - Elimina la base y sus cofres (en cascada).

ENTIDAD JUGADOR: `/api/players`

- * GET `/api/players` - Recupera todos los jugadores.
- * GET `/api/players/uuid/{uuid}` - Obtiene los datos del jugador por su UUID.
- * POST `/api/players` - Payload JSON (`name`, `uuid`) registra al jugador.
- * PUT `/api/players/{id}/active-base/{bid}` - Establece cuál es su base activa in-game.

ENTIDAD COFRES (STORAGES): `/api/storages`

- * GET `/api/storages/base/{bid}` - Recupera todos los cofres físicos de la base.
- * POST `/api/storages` - Registra coordenadas X, Y, Z de un cofre.
- * POST `/api/storages/sync` - Sincroniza la lista completa de ítems de un cofre.
- * DELETE `/api/storages/base/{bid}/coords?x=..&y=..&z=..` - Borra el cofre por coordenadas.

ENTIDAD AUDITORÍA E HISTORIALES:

- * GET `/api/player-activity-logs` - Devuelve la lista completa de auditorías.
- * POST `/api/player-activity-logs` - Registra una nueva entrada de actividad.
- * GET `/api/inventory-histories` - Devuelve el histórico de transacciones de ítems.

* POST /api/inventory-histories - Crea una entrada de variación de inventario.

7. TECNOLOGÍAS UTILIZADAS EN EL PROYECTO

El desarrollo integral de "Minecraft Inventory Viewer" hace uso de un conjunto de tecnologías modernas orientadas tanto al desarrollo de videojuegos en tiempo real como al despliegue de microservicios empresariales de backend:

TECNOLOGÍAS DEL MOD CLIENTE (MINECRAFT FORGE):

1. Java Development Kit (JDK) 17: Versión estable de Java utilizada como lenguaje y entorno de ejecución común para ambos proyectos.
2. Minecraft Forge (1.18.2 - Versión 40.3.0): API y cargador de mods que provee acceso al motor de Minecraft, buses de eventos del mundo y APIs gráficas 2D/3D.
3. Gradle 8.8: Herramienta de automatización de compilación empleada en el cliente para configurar dependencias del entorno Forge y empaquetar el mod en un JAR.
4. Google Gson: Librería de serialización y deserialización JSON utilizada para analizar payloads de red e interpretar las respuestas de la API.
5. Java HttpClient (java.net.http): Cliente HTTP nativo moderno de Java utilizado para el envío asíncrono y no bloqueante de peticiones en hilos secundarios.

TECNOLOGÍAS DEL SERVIDOR BACKEND (SPRING BOOT REST API):

1. Spring Boot 3.x: Framework de desarrollo empresarial en Java que simplifica el despliegue de microservicios web REST independientes y autoconfigurados.
2. Spring Data JPA: Capa de abstracción para persistencia de datos orientada a simplificar el acceso a bases de datos relacionales mediante repositorios.
3. Hibernate 6.x: Motor de mapeo objeto-relacional (ORM) encargado de traducir las clases Java a sentencias SQL y gestionar transacciones ACID de forma óptima.
4. MariaDB / MySQL: Motor de base de datos relacional de alto rendimiento donde se almacenan y estructuran de manera consistente las tablas del proyecto.
5. Apache Maven: Gestor de ciclo de vida de software y dependencias utilizado para compilar y empaquetar el servidor REST en un artefacto ejecutable.
6. MapStruct: Generador de mapeadores Java seguro y extremadamente rápido en tiempo de compilación, usado para convertir entre entidades e interfaces DTO.

7. Proyecto Lombok: Librería para automatizar la generación de constructores, patrones Builder y getters/setters mediante anotaciones sencillas.

8. GUÍA DE INSTALACIÓN, INICIALIZACIÓN Y VALIDACIÓN (MANUAL DE PRUEBAS)

REQUISITOS PREVIOS:

- * Java Development Kit (JDK) 17 o superior instalado y configurado en el PATH.
- * Gestor de bases de datos MariaDB o MySQL activo localmente (puerto 3306).
- * Cliente de Minecraft con cargador de mods Forge 1.18.2 (Forge 40.3.0).

PASOS PARA LEVANTAR EL PROYECTO:

1. PREPARACIÓN Y CONFIGURACIÓN DEL SERVIDOR (BACKEND):

- Abre tu consola SQL (o phpMyAdmin / DBeaver) y crea la base de datos vacía:

```
"CREATE DATABASE inventoryviewer;"
```

- Ve a "inventoryviewerAPI/src/main/resources/application.properties" y coloca

los datos de acceso de tu servidor SQL:

```
"spring.datasource.url=jdbc:mariadb://localhost:3306/inventoryviewer"
```

```
"spring.datasource.username=tu_usuario"
```

```
"spring.datasource.password=tu_contraseña"
```

- Sitúate en la terminal en la carpeta "inventoryviewerAPI" y compila y arranca:

".\mvnw spring-boot:run" (en Windows) o "./mvnw spring-boot:run" (en Linux/macOS)

- El microservicio levantará el puerto en: "http://localhost:8080".

2. COMPILACIÓN E INSTALACIÓN DEL MOD DE MINECRAFT (CLIENTE):

- Abre tu terminal en la carpeta "ModMC".

- Compila el mod ejecutando el script de Gradle:

```
".\gradlew build" (en Windows) o "./gradlew build" (en Linux/macOS)
```

- Busca el archivo ".jar" del mod dentro de la carpeta "ModMC/build/libs/".

- Cópialo y pégalo dentro de la carpeta de mods del juego Minecraft (por defecto "%appdata%/.minecraft/mods" en Windows o "~/.minecraft/mods" en Linux).

3. MANUAL DE PRUEBAS DE VALIDACIÓN IN-GAME:

- Inicia Minecraft con el perfil de Forge 1.18.2.

- Entra a tu partida y presiona la tecla 'B'.

- El mod detectará automáticamente que eres un usuario nuevo no registrado en

la base de datos: registrará tu perfil en MariaDB, te entregará la Varita de Inventario y te enviará un mensaje verde en el chat notificándote de ello.

- Toma la varita de tu inventario, apunta al aire y haz clic derecho para abrir el HUD rápido. Crea una base escribiendo el nombre y la descripción.

- Apunta con la varita a un cofre en el mundo y haz clic derecho sobre él. Aparecerá un resalte tridimensional brillante que confirma que el cofre ha sido enlazado con éxito a tu base en la base de datos MariaDB.
- Abre el cofre, deposita ítems (ej. Madera, Lingotes de Hierro, Manzanas) y cierra el cofre.
- Abre el Dashboard general (tecla 'B') y comprueba que en la pestaña de "Catálogo" y de "Cofres" se lista y suma la cantidad real de tus ítems en tiempo real.
- Haz clic sobre la Manzana y verifica que la navegación cruzada te redirige al Wiki de Alimentos para darte información de nutrición y saturación.
- ¡Tu sistema Minecraft Inventory Viewer está 100% funcional y listo!

=====